

FIT2109 Computer Science Workshop

Week 2: Shell Tools & Scripting

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

From Week 1 to reliable automation

Week 1 gave us the control surface

We can navigate, inspect files, run commands, read permissions, and reason about streams.

This week is about precision

The shell rewrites command lines before running them. Quoting, variables, regexes, exit status, and safety flags decide whether automation is reliable or fragile.

Bash is everywhere, and small mistakes matter

Research finding

A study of real open-source Bash scripts found common bugs around quoting, word splitting, path management, command options, return values, and error handling.

- These are not obscure edge cases; they appear in widely used projects.
- The shell rewards careful habits and punishes silent assumptions.
- FIT2109 uses Bash scripts even when your interactive shell is Zsh.

Reference

Bash in the Wild: Language Usage, Code Smells, and Bugs — Yiwen Dong, Zheyang Li, Yongqiang Tian, et al. ACM Transactions on Software Engineering and Methodology (TOSEM), 2022.

A short history of Bash

- Unix shells began as command-line interfaces for the operating system.
- The Bourne shell, `sh`, became an early standard Unix shell.
- Bash means **B**ourne **A**gain **S**hell.
- Bash is both an interactive command interface and a scripting language.
- The same evaluation rules apply when you type commands and when a script runs them.

Practical consequence

Learning Bash is not just learning command names. It is learning how the shell interprets text.

Learning goals

By the end of this workshop, you should be able to:

- Predict how quoting, variables, globs, and splitting affect a command.
- Use `export`, `env`, `$?`, `&&`, and `||`.
- Distinguish shell globs from regular expressions.
- Build `grep`, `find`, `sort`, `uniq`, and `wc` pipelines.
- Review and improve Bash scripts using arguments, conditionals, loops, simple functions, and safe quoting.
- Connect to a remote server with `ssh`, transfer files with `scp`, and explain why SSH keys replace passwords.

The shell rewrites before it runs

Shell evaluation model

The shell processes a command line in stages.

- 1 Read the input line.
- 2 Parse quoting and structure.
- 3 Expand variables, globs, and command substitutions.
- 4 Split unquoted results on whitespace.
- 5 Execute the resulting command with its arguments.

Key idea

What you type is not always what the command receives.

Quoting controls what the shell changes

No quotes

Variables expand, then split on spaces.

```
file="Week 2.txt"
```

```
wc -l $file
```

Shell passes two filenames:
Week and 2.txt

Double quotes

Variables expand, but spaces stay inside one argument.

```
wc -l "$file"
```

Shell passes one filename:
Week 2.txt

Single quotes

Text is literal. No variable expansion.

```
wc -l '$file'
```

Shell looks for a file named:
\$file

Professional habit

Quote variables that may contain spaces or special characters: "\$file", "\$1".

Variables can stay local or travel

Shell variable

Visible in the current shell.

```
course=FIT2109  
echo "$course"
```

Environment variable

Inherited by child processes after export.

```
export course  
bash -c 'echo $course'
```

Per-command environment

```
COURSE=FIT2109 env | grep '^COURSE='
```

Command substitution

```
today=$(date +%F) — run a command and capture its output in a variable.
```

Exit status lets commands make decisions

Key idea

Every command returns an exit status. 0 usually means success; nonzero means failure.

- `$?`: exit status of the previous command.
- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeds.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` fails.

Common script pattern

```
cd "$dir" || exit 1
```

Demo 1: shell evaluation

Predict first

Before watching, agree with your neighbours: how many arguments does `ls $file` receive when the filename contains spaces? What does a child shell see before and after `export`?

Watch

Compare unquoted and quoted variables, then local variables and exported variables.

Note

The most important question is not "what command was typed?" It is "what arguments did the command receive?"

Demo 2: exit status as control flow

Predict first

`mkdir demo_dir` runs twice. Before watching: what does `echo $?` print after the second run?

Watch

Use `$?`, `&&`, and `||` to make the shell react to success and failure.

Pause and reason

Why should a script stop if `cd "$dir"` fails?

Globs and regexes are different languages

Shell glob

Expanded by the shell before the command runs. Matches filenames.

`*.log`

Regular expression

Interpreted by a tool. Matches text.

`.*\.log$`

Common mistake

Do not confuse a filename pattern with a text pattern. Quote regexes so the shell does not change them first.

grep flags shape the question

- -E: extended regex.
- -o: print only the matched text.
- -n: show line numbers.
- -q: quiet; exit status only.
- -r: search recursively.
- -i: case-insensitive.

Useful patterns

`grep -E 'WARN(ING)?'` matches `WARN` and `WARNING`.

`grep -Eo 'ERROR [0-9]+'` extracts just the error code.

Text tools compose into pipelines

Key idea

Each command should do one small job, then pass text to the next command.

- `grep`: select matching lines or substrings.
- `sort`: put equal values together.
- `uniq -c`: count adjacent repeated lines.
- `wc -l`: count lines.

Pipeline reasoning

Always explain a pipeline one stage at a time.

find searches by file metadata

Key idea

`find` locates files using filesystem properties: name, type, permissions, and full paths.

- `-name '*.log'`: filename pattern. Quote the glob.
- `-type f`: regular files only.
- `! -perm -u=x`: files not executable by owner.
- `-regex`: regex over the full path.
- `-exec cmd {} +`: run a command on matching files.

Demos 3–4: grep, pipelines, and find

Watch

Start from a concrete question about logs and source files, then build the command piece by piece.

Note

The hard part is choosing the right question: filename search, text search, counting, or metadata filtering.

Pause and reason: full-path matching

Question

Why might a `find -regex` pattern that looks correct fail unless it accounts for the leading path?

Hint

`find -regex` does not only match the final filename.

Handout Activity 1: log investigation

Now work on Activity 1 in the handout

Your group has a server log and real questions to answer: which errors occur, and which occur most often? Design the commands, build the pipeline, and defend each stage.

Group output

Complete Parts A–B: answer the log questions with `grep` and a counting pipeline, then track down files with `find`.

Scripts make workflows reusable

Key idea

A shell script is a text file that Bash reads and evaluates, just as if you typed each command yourself.

- Shebang: `#!/usr/bin/env bash`
macOS users: this ensures Bash runs the script even if your interactive shell is Zsh.
- Positional arguments: `$1`, `$2`, ...
- Argument count: `$#` — check it first: `[ "$#" -eq 0 ]`
- Default value: `${1:-world}`
- Conditional: `if`, `then`, `else`, `fi`
- File tests: `[ -f "$file" ]` for a file, `[ -d "$dir" ]` for a directory.
- Loop: `for x in ...; do ...; done`

Bash can group commands into functions

Recognition pattern

A Bash function names a small command sequence so the script can call it more than once.

```
warn() {  
    echo "warning: $1" >&2  
}  
  
warn "missing input"
```

What does \$1 mean?

In a Bash function, \$1 is the first argument passed to the function call. Here, `warn "missing input"` makes \$1 equal to `missing input`.

Reliable scripts fail clearly

Three safety habits

- `set -u`: fail on unset variables.
- `set -o pipefail`: fail if any pipeline stage fails.
- Quote variables: `"$1"`, `"$file"`.

Why this matters

Silent empty strings and hidden pipeline failures can make a script appear successful when it did the wrong thing.

Useful debugging trace

`bash -x script.sh` or `set -x` prints each command after expansion, which is handy for showing what the shell actually runs. ShellCheck (shellcheck.net) reviews scripts for these pitfalls automatically.

Demos 5–6: script mechanics and silent failures

Watch

Demo 5: make a small script executable and pass it arguments. Demo 6: a script fails *silently* — expose the bug with `bash -x`, then make it fail loudly with `set -u`.

Note

The demo shows how to *find* a hidden failure. In Activity 2, your group finds and fixes the failures in a different script.

Demo 6 continued: loop over log files

Watch

Turn a one-file command into a reusable loop over many log files.

Pause and reason

If no `.log` file matches the glob, what does the loop variable receive — and what happens then?

Handout Activity 2: script review and repair

Today's main group task (about 25 minutes)

Review a fragile script, identify failure modes, and rewrite it using safer Bash habits — this is where everything from today comes together.

Group output

Complete Parts A–D: bug list, safer rewrite, safety-flag explanation, and loop design.

What is SSH?

SSH means Secure Shell

SSH is a protocol for securely connecting to another machine over a network. It encrypts the connection so you can log in, run commands, and transfer files safely.

- It is the standard way to work with remote Linux servers.
- It supports both interactive sessions and one-off remote commands.
- Your local terminal stays the same; the commands run on the other machine.

Logging in with a password

The simplest way in

```
ssh username@fit2109s2.rep.monash.edu
```

- On the first connection, SSH asks you to trust the server's fingerprint.
- Then the server asks for your account password.
- You now have a shell *on the remote machine*; type `exit` to come back.
- `ssh user@host 'command':` run one remote command.
- `scp local user@host:/path/:` copy to remote.
- `scp user@host:/path/file .:` copy from remote.

Do you want to type your password every time?

The friction

Every `ssh` and every `scp` asks for your password again. That is tedious for you, and automation cannot type passwords for you.

Use an SSH key!

Set it up once, then log in without typing a password:

- `ssh-keygen -t ed25519`: generate a key pair on your machine.
- `ssh-copy-id user@host`: install the public key on the server (last password prompt!).
- `ssh user@host`: connects without a password from now on.

SSH keys separate public and private trust

Private key

Stays on your machine. Never share it.
Protect it like a password.

Public key

Copied to the remote server, usually into
`~/.ssh/authorized_keys`.

How the server decides to let you in

The server grants access when your private key matches a public key it trusts. To revoke access, the server just removes that public key — no password to change or share.

Demo 7: remote workflow

Watch

Connect with a password and feel the friction. Then generate a key pair, install the public key, and connect again — no password. Finish by running one remote command and transferring a file.

Pause and reason

Which file was copied to the server, and why is it safe to copy that one?

Final checkpoint: Poll Everywhere



pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to pollev.com/fit2109.

Purpose

Use the Poll Everywhere questions to check which Week 2 ideas need one more example before we finish.

What should feel different after today?

- You can explain why quoting changes command behaviour.
- You can use exit status to make shell workflows branch safely.
- You can distinguish globs, regexes, text search, and file search.
- You can build a pipeline to answer a question from text.
- You can review a script for quoting, missing input, and hidden failures.
- You can describe how SSH keys support remote command-line workflows.

- ❶ Why do `$file` and `"$file"` sometimes pass a different number of arguments?
- ❷ What is the difference between a shell glob and a regular expression?
- ❸ What does `cd "$dir" || exit 1` protect a script against?
- ❹ What do `set -u` and `set -o pipefail` each catch?
- ❺ After `ssh-copy-id`, why does `ssh` stop asking for your password?

Reflection: Which Bash habit from today do you most want to make automatic?

Questions?

Before the applied class

Practise explaining not just what a command does, but why it is written that way. On Ed, read and run the worked `log-report` example, then try the `warn-summary` coding challenge.